FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Jullianne Gabrielle G. Magbanua
Section:	AN41
Professor:	Ms. Crisola Tan
Date:	Oct. 3, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

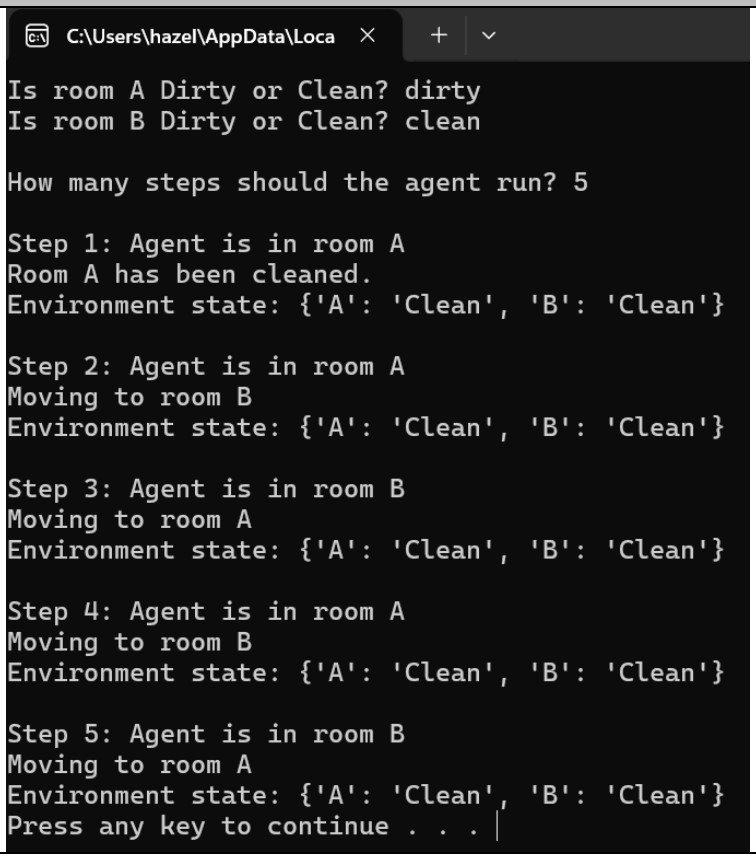
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

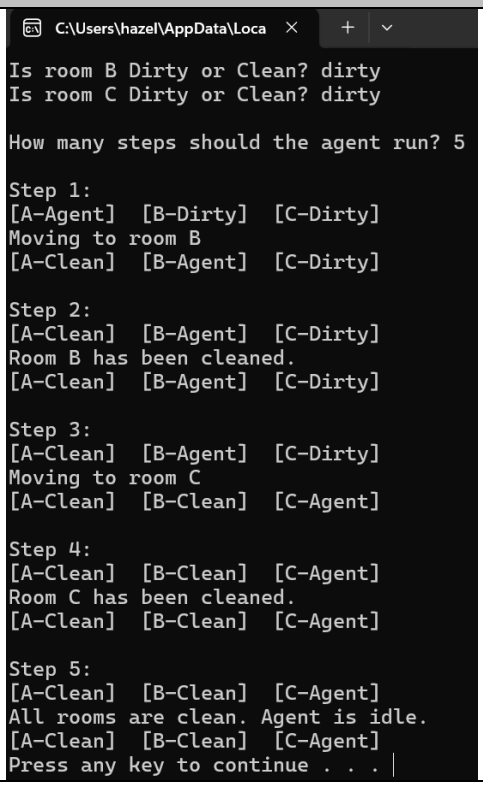
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
import random

# Environment
rooms = {
    "A": input("Enter Room A state (Dirty/Clean): ").capitalize(),
    "B": input("Enter Room B state (Dirty/Clean): ").capitalize(),
    "C": input("Enter Room C state (Dirty/Clean): ").capitalize()
```

```
}


def is_dirty(room):
    """Check if a room is dirty."""
    return rooms[room] == "Dirty"


def clean_room(room):
    """Clean the specified room."""
    rooms[room] = "Clean"


# Bonus 4: Text-Based Visualization
def display_grid(agent):
    print("\nROOM MAP")

    for room in ["A", "B", "C"]:
        if room == agent.location:
            print(f"[{room}: {rooms[room]} 🤖]", end=" ")
        else:
            print(f"[{room}: {rooms[room]}]", end=" ")

    print("\n")


# Rule-Based Agent
class VacuumAgent:
    def __init__(self):
        self.location = "A"

    # Three-room movement
    def move(self):
        if self.location == "A":
            self.location = "B"
        elif self.location == "B":
            self.location = "C"
        else:
            self.location = "A"

    # Randomly choose a dirty room
    def perceive_and_act(self):
```

```
        # Rule 1: Clean current room if dirty
        if is_dirty(self.location):
            clean_room(self.location)
            return f"Cleaned Room {self.location}"

        # Find all dirty rooms
        dirty_rooms = []

        for room in rooms:
            if rooms[room] == "Dirty":
                dirty_rooms.append(room)

        # Move to a random dirty room if any exist
        if dirty_rooms:
            self.location = random.choice(dirty_rooms)
            return f"Moved to dirty Room {self.location}"

        # Otherwise continue normal movement
        self.move()
        return f"Moved to Room {self.location}"


# Create the agent
agent = VacuumAgent()

# Run the Simulation
steps = int(input("Enter number of steps: "))

print("\nVACUUM CLEANER SIMULATION")
print("=" * 35)

for step in range(1, steps + 1):

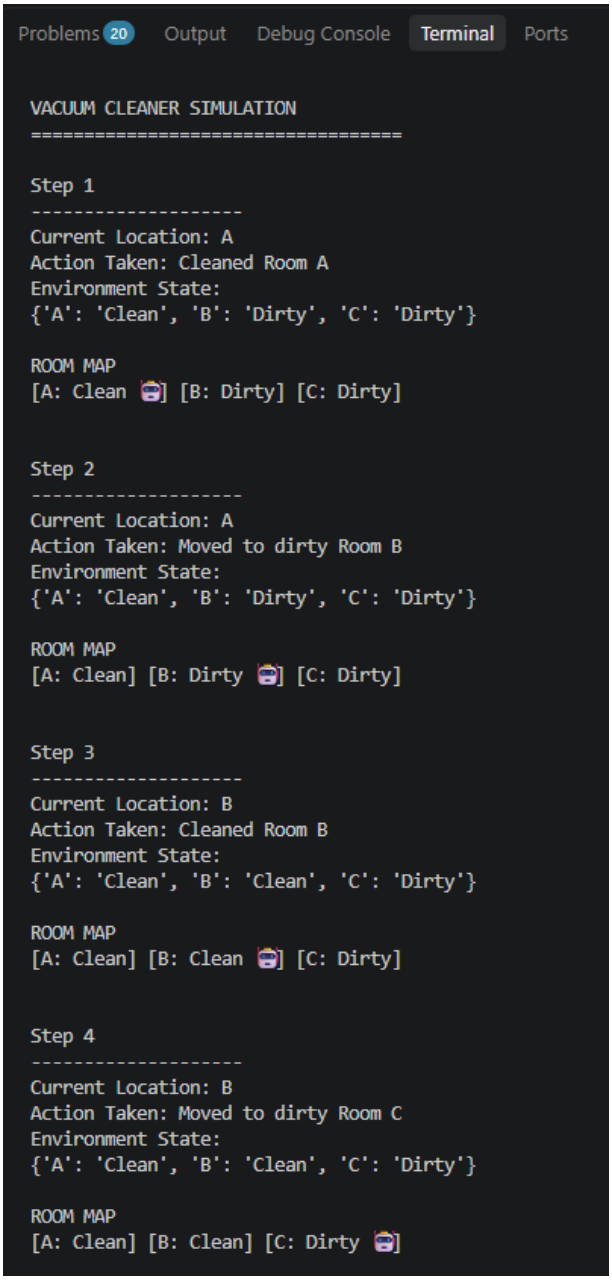
    print(f"\nStep {step}")
    print("-" * 20)
    print(f"Current Location: {agent.location}")

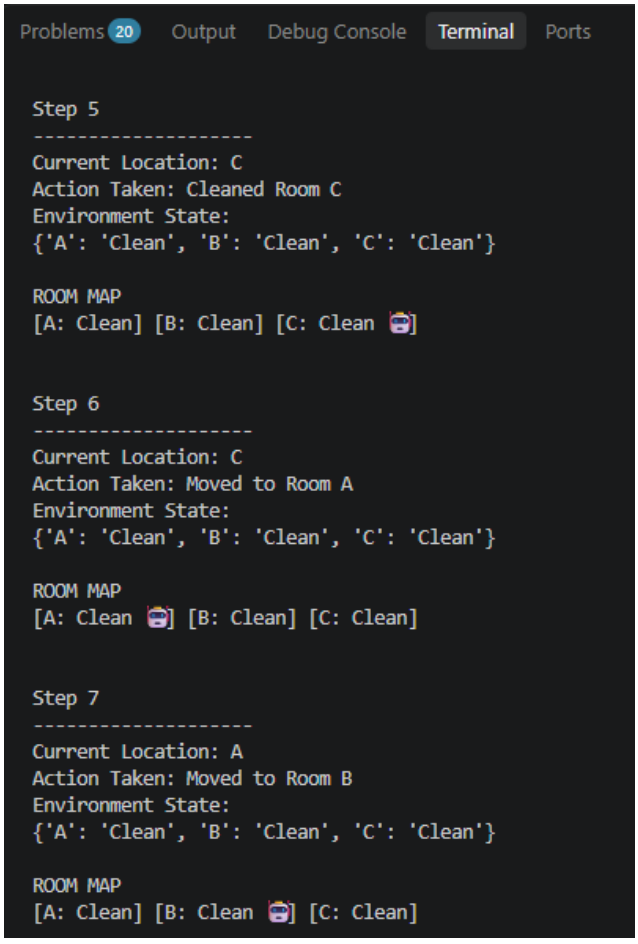
    action = agent.perceive_and_act()

    print(f"Action Taken: {action}")
    print("Environment State:")
    print(rooms)
```

```
    # Visualize agent position
    display_grid(agent)
```

OUTPUT:





Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. The agent follows a set of predefined rules. If the current room is dirty, it cleans the room. If the current room is clean, it moves to another room. In the bonus version, the agent prioritizes moving to a dirty room if one is available.
2. The agent first checks the condition of its current room. If the room is dirty, it performs the cleaning action. If the room is already clean, it searches for dirty rooms and moves to one of them. If no dirty rooms remain, it follows its normal movement pattern between rooms.
3. If both rooms are clean at the beginning of the simulation, the agent does not perform any cleaning actions. Instead, it continuously moves between the rooms because there are no dirty rooms that require cleaning.
4. The agent can be modified by adding additional rooms to the environment and updating the movement logic to allow navigation among all available rooms. In our bonus implementation, a third room (Room C) was added, and the agent was enhanced to identify and move to dirty rooms before following its regular movement pattern. This makes the agent more flexible and efficient in larger environments.